A3 · Name the Layer

CS425 Computer Networks · in class activity · graded pass/fail

TURN THIS IN ON PAPER
before you leave the room
bring your A2 station table

Names _____ Date _____

_____ _____

TARGET CARD · these are NEW addresses, not your A2 ones

ALPHA = _____ BRAVO = _____

ROUND 1 · BUILD THE DECISION TREE

This is the part you keep. Every leaf names a layer and names who can fix it: you, the person running the server, or the network in between. Glossary on the back if a term is unfamiliar.

START · "it does not work". First question, because if the answer is bad you never sent a single packet to the target:
_____

No answer / NXDOMAIN.
cause _____ layer _____ fixed by _____

An answer comes back, but it is an address you cannot route to.
cause _____ layer _____ fixed by _____

An answer comes back and it is routable. Next question: _____

Fails instantly. The word your tool uses is _____. The host sent back a _____.
cause _____ layer _____ fixed by _____

Hangs, then gives up after several seconds. What came back? _____
cause _____ layer _____ fixed by _____

The connection completes. Last question: _____

Open, but no bytes ever arrive.
cause _____ layer _____ fixed by _____

It works. If ping *still* fails, that tells you _____

ROUND 1 · SUMMARY TABLE

One row per station from A2, so your tree and your observations agree. A row that fits no leaf means your tree is missing a branch. That is a finding, not a mistake.

STA	WHAT YOU SAW IN A2	WHAT IT MEANS ABOUT WHAT THE OTHER END DID	LAYER	WHO FIXES IT
1				
2				
3				
4				
5				
6				
7				

ROUND 2 · THE FAILURE YOUR TREE PROBABLY MISSES

Station 7 is ALPHA port 8084. Probe it the way you probed everything in A2.

```
ALPHA=203.0.113.10          # replace with YOUR alpha address from the card above
nc -vz -w 5 $ALPHA 8084     # macOS: -G 5 instead of -w 5
curl -sS -o /dev/null -m 8 -w 'connect=%{time_connect} total=%{time_total}\n' http://$ALPHA:8084/
```

#	TARGET	NC SAID	CURL PRINTED, WORD FOR WORD	CONNECT=
7	ALPHA 8084			

1. Which layer completed successfully, and which one never did?

2. What is `connect=` here, and what was it for station 2? What does that difference tell you?

3. A load balancer decides whether a server is healthy by opening a TCP connection to its port. What does that health check say about this server, and why is that **worse** than having no health check at all?

4. Where does this land on the tree you built in Round 1? If it does not land anywhere, add the branch and say so here.

ROUND 3 · PREDICT, THEN WATCH

For station 2 in A2 your packets vanished. Did they vanish before they reached the host, or after? You cannot answer that from your laptop. Commit to an answer in writing before anything is run: a prediction you got wrong is full credit, a blank box is not.

Predict, before the demo

1. Do you expect the packets reached ALPHA or not? Why?

2. What command would you run *on the server* to find out? You want a tool that shows packets arriving on an interface.

Watch, during the demo

3. What appeared for **port 8081**?

4. What appeared for **port 8082**?

5. Explain the difference. One port shows you a conversation. The other shows you nothing at all, and "nothing at all" is the evidence. Evidence of what?

ROUND 4 · SAY IT OUT LOUD

1. Give the two symptom words that a hang and a refusal produce, and say what the server sends in each case. One of them sends nothing at all.

2. **You are on call.** Somebody reports "our API is down". You get *one* command before you have to say something useful. Which single command do you run, and what are the three different things its output can tell you? Defend the choice.

Graded pass/fail on a serious attempt at every round. Hand this in before you leave, with every name at the top.

THE FOUR LAYERS

Application
Do the useful thing. HTTP, SMTP, DNS.

Transport
Bytes to the right *program*. TCP, UDP.

Network
A packet to the right *host*. IP, ICMP.

Link
A frame to the next hop on this wire. Ethernet, Wi-Fi.

NAMES

DNS
The application layer protocol that turns a name into an IP address. A real query to a real server, so it can fail like anything else.

Authoritative
Holding the real records rather than a cached copy. An authoritative "no" is a fact.

NXDOMAIN
That name does not exist. An answer, not an error. You have sent **zero** packets to your target, so this is not a connectivity problem. `dig +short` hides it as an empty line.

ADDRESSES

IP address
Network layer identifier for an *interface*, not a machine and not a user.

Private address, RFC 1918
`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. **Not routable on the public internet.**

NAT
Rewrites addresses and ports so many private machines share one public address.

CONNECTIONS

Port
A 16 bit number saying *which program* gets the data. The address finds the machine, the port finds the process.

Listening socket
A socket a server is waiting on. No listener means the host has nobody to hand an arriving connection to.

Three way handshake
SYN, SYN-ACK, ACK. `connect()` returns when the SYN-ACK arrives, so connecting costs **one round trip** before any of your data moves.

SYN
The first packet of the handshake. If you see one arrive and nothing sensible come back, the host received your attempt.

RST
A TCP flag meaning "nothing here, stop", sent when a connection arrives for a port with no listener. Getting one **proves** the address is routable, the host is up, and its stack is running.

Connection refused
What a client reports after a RST. About one round trip, so effectively instant.

Connection timeout
What a client reports after **nothing at all**. It retransmits with exponential backoff and gives up on its own clock.

Half open / stalled
The handshake finished and the application never wrote. Transport is healthy, the service is not, and a port check cannot tell.

FILTERING

Firewall, packet filter
Anything that inspects packets and decides whether they pass, usually on addresses and ports.

DROP versus REJECT
DROP discards silently; the sender waits out its own timeout. **REJECT** sends back an error, usually a RST. From the client, a dropped packet and a host that does not exist look identical.

Security group
AWS's packet filter, attached to the virtual interface rather than the host. It sits **in front of** the machine, so a blocked packet never reaches the OS and a capture on the host shows nothing.

Host firewall
A filter running *on* the machine, such as `iptables`. A capture on the host *does* see these packets arrive, which is how you tell the two apart.

TOOLS

`nc -vz`
Is this TCP port open? Names the failure clearly: `Connection refused` versus `Operation timed out`. Connect timeout is `-w` on Linux and `-G` on macOS; the wrong one waits about 75 seconds.

`curl -v`
Walks DNS, TCP and HTTP in one command and tells you where it stopped.

`ss -ltn`
What is listening on this machine? Run it on the server.

`tcpdump`
What packets are actually arriving? `sudo tcpdump -nni any tcp port 8081`. Use `-nn` so it does not rename ports.

`connect=`
`0.000000` means the handshake never completed. A real value followed by a timeout means TCP succeeded and the *application* never answered.